

Mutchumari Chandrsekhar Reddy

AP25110010976

1. Implementation of finding the maximum and minimum element using divide and conquer strategy.

Program:

```
#include <stdio.h>
```

```
void maxmin(int a[], int low, int high, int *max, int *min) { int mid, max1, min1, max2, min2;
```

```
if (low == high)
{
    *max = *min = a[low];
    return;
}
```

```
if (high == low + 1)
{
    if (a[low] > a[high])
    {
        *max = a[low];
        *min = a[high];
    }
    else
    {
        *max = a[high];
        *min = a[low];
    }
    return;
}
```

```
mid = (low + high) / 2;
```

```
maxmin(a, low, mid, &max1, &min1);
maxmin(a, mid + 1, high, &max2, &min2);
```

```

*max = max1 > max2 ? max1 : max2;
*min = min1 < min2 ? min1 : min2;
}

int main() { int a[100], n, i, max, min;

printf("Enter number of elements: ");
scanf("%d", &n);

printf("Enter elements: ");
for (i = 0; i < n; i++)
    scanf("%d", &a[i]);

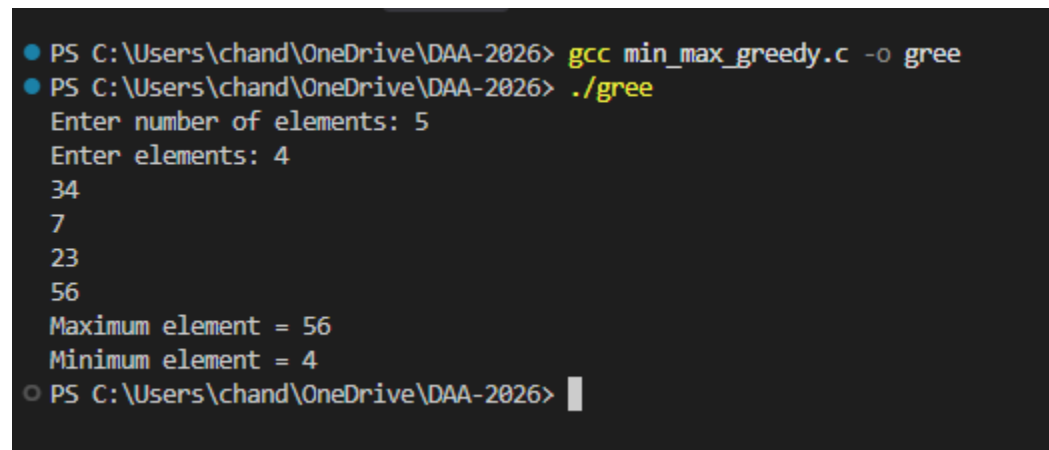
maxmin(a, 0, n - 1, &max, &min);

printf("Maximum element = %d\n", max);
printf("Minimum element = %d\n", min);

return 0;
}

```

Terminal Process:



```

PS C:\Users\chand\OneDrive\DAA-2026> gcc min_max_greedy.c -o gree
PS C:\Users\chand\OneDrive\DAA-2026> ./gree
Enter number of elements: 5
Enter elements: 4
34
7
23
56
Maximum element = 56
Minimum element = 4
PS C:\Users\chand\OneDrive\DAA-2026>

```

2. Divide and conquer: Implementation of maximum-subarray problem.

Program:

```
#include <stdio.h>
```

```

int max(int a, int b)
{
    return a > b ? a : b;
}

int maxCrossingSum(int a[], int low, int mid, int high)
{
    int sum = 0, leftSum = -999999, rightSum = -999999;
    int i;
    for (i = mid; i >= low; i--)
    {
        sum += a[i];
        if (sum > leftSum)
            leftSum = sum;
    }
    sum = 0;
    for (i = mid + 1; i <= high; i++)
    {
        sum += a[i];
        if (sum > rightSum)
            rightSum = sum;
    }
    return leftSum + rightSum;
}

```

```

int maxSubarray(int a[], int low, int high)
{
    int mid, left, right, cross;
    if (low == high)

```

```

        return a[low];
    mid = (low + high) / 2;
    left = maxSubarray(a, low, mid);
    right = maxSubarray(a, mid + 1, high);
    cross = maxCrossingSum(a, low, mid, high);
    return max(max(left, right), cross);
}

int main()
{
    int a[100], n, i, result;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    result = maxSubarray(a, 0, n - 1);
    printf("Maximum subarray sum = %d\n", result);
    return 0;
}

```

Terminal Process:

```

● PS C:\Users\chand\OneDrive\DAA-2026> gcc sub.c -o sub
● PS C:\Users\chand\OneDrive\DAA-2026> ./sub
Enter number of elements: 5
Enter elements: 6
43
56
34
67
Maximum subarray sum = 206
● PS C:\Users\chand\OneDrive\DAA-2026>

```

3. Divide and conquer: Implementation of Strassen's algorithm for matrix multiplication.

Program:

```
#include <stdio.h>

void strassen(int A[2][2], int B[2][2], int C[2][2])
{
    int M1, M2, M3, M4, M5, M6, M7;
    M1 = (A[0][0] + A[1][1]) * (B[0][0] + B[1][1]);
    M2 = (A[1][0] + A[1][1]) * B[0][0];
    M3 = A[0][0] * (B[0][1] - B[1][1]);
    M4 = A[1][1] * (B[1][0] - B[0][0]);
    M5 = (A[0][0] + A[0][1]) * B[1][1];
    M6 = (A[1][0] - A[0][0]) * (B[0][0] + B[0][1]);
    M7 = (A[0][1] - A[1][1]) * (B[1][0] + B[1][1]);
    C[0][0] = M1 + M4 - M5 + M7;
    C[0][1] = M3 + M5;
    C[1][0] = M2 + M4;
    C[1][1] = M1 - M2 + M3 + M6;
}

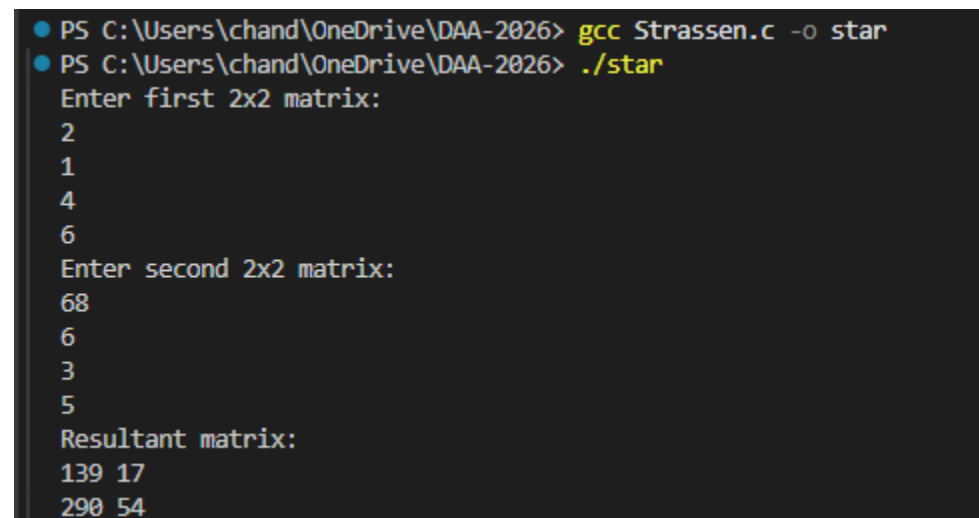
int main()
{
    int A[2][2], B[2][2], C[2][2];
    int i, j;
    printf("Enter first 2x2 matrix:\n");
    for (i = 0; i < 2; i++)
        for (j = 0; j < 2; j++)
            scanf("%d", &A[i][j]);
```

```

printf("Enter second 2x2 matrix:\n");
for (i = 0; i < 2; i++)
    for (j = 0; j < 2; j++)
        scanf("%d", &B[i][j]);
strassen(A, B, C);
printf("Resultant matrix:\n");
for (i = 0; i < 2; i++)
{
    for (j = 0; j < 2; j++)
        printf("%d ", C[i][j]);
    printf("\n");
}
return 0;
}

```

Terminal process:



```

PS C:\Users\chand\OneDrive\DAA-2026> gcc Strassen.c -o star
PS C:\Users\chand\OneDrive\DAA-2026> ./star
Enter first 2x2 matrix:
2
1
4
6
Enter second 2x2 matrix:
68
6
3
5
Resultant matrix:
139 17
290 54

```

4. Greedy Approach: Implementation of Fractional Knapsack.

Program:

```
#include <stdio.h>

struct Item
{
    int weight;
    int value;
    float ratio;
};

int main()
{
    struct Item item[100], temp;
    int n, capacity, i, j;
    float total = 0;
    printf("Enter number of items: ");
    scanf("%d", &n);
    printf("Enter weight and value of each item:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d %d", &item[i].weight, &item[i].value);
        item[i].ratio = (float)item[i].value / item[i].weight;
    }
    printf("Enter capacity of knapsack: ");
    scanf("%d", &capacity);
    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
```

```

        if (item[i].ratio < item[j].ratio)
        {
            temp = item[i];
            item[i] = item[j];
            item[j] = temp;
        }
    }
}

for (i = 0; i < n; i++)
{
    if (capacity >= item[i].weight)
    {
        capacity -= item[i].weight;
        total += item[i].value;
    }
    else
    {
        total += item[i].ratio * capacity;
        break;
    }
}

printf("Maximum profit = %.2f\n", total);
return 0;
}

```

Terminal process:

```
● PS C:\Users\chand\OneDrive\DAA-2026> gcc kanp.c -o kn
● PS C:\Users\chand\OneDrive\DAA-2026> ./kn
Enter number of items: 5
Enter weight and value of each item:
12 4
13 7
15 6
18 4
20 6
Enter capacity of knapsack: 20
Maximum profit = 9.80
○ PS C:\Users\chand\OneDrive\DAA-2026> █
```